

Universidad Internacional del Ecuador

Nombre: Gabriel Eduardo Rios Fernández

Fecha: 20.5.2026

Enlace del github: <https://github.com/gariosfe/archivos-para-git.git>

Definición de contratos y debate técnico de protocolos (REST vs. gRPC)

Introducción

El sistema de Reserva de Citas Médicas es un escenario donde dos servicios intercambian información sensible de manera síncrona: el Portal del Paciente (Servicio A) y el Motor de Agenda Clínica (Servicio B). El paciente solicita una cita, el portal valida disponibilidad en tiempo real y confirma o rechaza la reserva, todo dentro de la misma transacción HTTP

Justificación de la Elección

Este caso posee cuatro características que lo convierten en un banco de pruebas ideal para comparar REST y gRPC:

Característica	Impacto en el protocolo
Datos sensibles (FHIR, PII)	Requiere contratos estrictos y tipado fuerte
Interacción síncrona (request/reply)	El cliente espera la respuesta antes de continuar
Baja tolerancia a la latencia	El paciente no puede esperar > 2 s en la UI
Alta disponibilidad requerida	Las falacias de la computación distribuida son críticas

Actores del Sistema

Actor / Componente	Rol	Tecnología supuesta
Portal del Paciente (Svc A)	Inicia la reserva; interfaz de usuario	React + Node.js
Motor de Agenda (Svc B)	Gestiona disponibilidad de médicos y slots	Java Spring Boot
Base de Datos Clínica	Almacén de citas, horarios, pacientes	PostgreSQL
Servicio de Notificaciones	Envía confirmación por correo/SMS	Python (async)

Especificación Técnica — REST (OpenAPI 3.0)

Diseño de la Interfaz REST

El contrato REST modela los recursos del dominio médico (pacientes, médicos, citas) como URLs navegables. Las operaciones CRUD se mapean a verbos HTTP estándar (POST para crear, GET para leer). La respuesta sigue el estándar RFC 7807 Problem Details para errores.

Endpoints Principales

Método	Endpoint	Descripción
POST	/v1/appointments	Crea una nueva reserva de cita
GET	/v1/appointments/{id}	Consulta estado de una cita
GET	/v1/doctors/{id}/slots	Lista slots disponibles de un médico
DELETE	/v1/appointments/{id}	Cancela una cita existente
PATCH	/v1/appointments/{id}	Modifica fecha/hora de la cita

Definición del Contrato — openapi.yaml

```
OPENAPI: "3.0.3"
INFO:
  TITLE: "MEDICAL APPOINTMENT SCHEDULING API"
  VERSION: "1.0.0"
  DESCRIPTION: |
    API REST PARA LA RESERVA SÍNCRONA DE CITAS MÉDICAS.
    INTERCAMBIA INFORMACIÓN SENSIBLE ENTRE EL PORTAL DEL
    PACIENTE (SVC-A) Y EL MOTOR DE AGENDA CLÍNICA (SVC-B).
  CONTACT:
    NAME: "EQUIPO DE INTEGRACIÓN"
    EMAIL: "INTEGRACION@CLINICA.COM"
  SERVERS:
    - URL: "HTTPS://API.CLINICA.COM/V1"
      DESCRIPTION: "PRODUCCIÓN"
    - URL: "HTTPS://STAGING-API.CLINICA.COM/V1"
      DESCRIPTION: "STAGING / QA"

PATHS:
  /APPOINTMENTS:
    POST:
      SUMMARY: "CREAR UNA NUEVA CITA MÉDICA"
      OPERATIONID: "CREATEAPPOINTMENT"
      TAGS: ["APPOINTMENTS"]
      REQUESTBODY:
        REQUIRED: TRUE
        CONTENT:
          APPLICATION/JSON:
            SCHEMA:
              $REF: "#/COMPONENTS/SCHEMAS/APPOINTMENTREQUEST"
      EXAMPLE:
        PATIENTID: "PAT-20045"
        DOCTORID: "DOC-0082"
        SPECIALTYCODE: "CARDIO"
```

REQUESTEDSLOT: "2025-08-14T10:00:00-05:00"
REASON: "CONTROL POST-OPERATORIO"
INSURANCEID: "INS-77231"
RESPONSES:
"201":
DESCRIPTION: "CITA CREADA EXITOSAMENTE"
HEADERS:
LOCATION:
SCHEMA:
TYPE: STRING
DESCRIPTION: "URL DEL RECURSO RECIÉN CREADO"
CONTENT:
APPLICATION/JSON:
SCHEMA:
\$REF: "#/COMPONENTS/SCHEMAS/APPOINTMENTRESPONSE"
"400":
\$REF: "#/COMPONENTS/RESPONSES/BADREQUEST"
"409":
\$REF: "#/COMPONENTS/RESPONSES/CONFLICT"
"503":
\$REF: "#/COMPONENTS/RESPONSES/SERVICEUNAVAILABLE"

/APPOINTMENTS/{APPOINTMENTID}:

GET:
SUMMARY: "OBTENER ESTADO DE UNA CITA"
OPERATIONID: "GETAPPOINTMENT"
TAGS: ["APPOINTMENTS"]
PARAMETERS:
- NAME: APPOINTMENTID
IN: PATH
REQUIRED: TRUE
SCHEMA:
TYPE: STRING
PATTERN: "^APT-[0-9]{8}\$"
RESPONSES:
"200":
DESCRIPTION: "DETALLE DE LA CITA"
CONTENT:
APPLICATION/JSON:
SCHEMA:
\$REF: "#/COMPONENTS/SCHEMAS/APPOINTMENTRESPONSE"
"404":
\$REF: "#/COMPONENTS/RESPONSES/NOTFOUND"

/DOCTORS/{DOCTORID}/SLOTS:

GET:
SUMMARY: "LISTAR SLOTS DISPONIBLES DE UN MÉDICO"
OPERATIONID: "GETDOCTORSLOTS"
TAGS: ["AVAILABILITY"]
PARAMETERS:
- NAME: DOCTORID
IN: PATH
REQUIRED: TRUE
SCHEMA:

TYPE: STRING
- NAME: DATE
IN: QUERY
REQUIRED: TRUE
SCHEMA:
TYPE: STRING
FORMAT: DATE
RESPONSES:
"200":
DESCRIPTION: "LISTA DE SLOTS DISPONIBLES"
CONTENT:
APPLICATION/JSON:
SCHEMA:
\$REF: "#/COMPONENTS/SCHEMAS/SLOTLIST"

COMPONENTS:

SCHEMAS:

APPOINTMENTREQUEST:

TYPE: OBJECT
REQUIRED:
- PATIENTID
- DOCTORID
- SPECIALTYCODE
- REQUESTEDSLOT

PROPERTIES:

PATIENTID:
TYPE: STRING
DESCRIPTION: "IDENTIFICADOR ÚNICO DEL PACIENTE"
EXAMPLE: "PAT-20045"

DOCTORID:
TYPE: STRING
DESCRIPTION: "IDENTIFICADOR ÚNICO DEL MÉDICO"
EXAMPLE: "DOC-0082"

SPECIALTYCODE:
TYPE: STRING
ENUM: [CARDIO, NEURO, PEDIA, TRAUMA, DERM]

REQUESTEDSLOT:
TYPE: STRING
FORMAT: DATE-TIME
DESCRIPTION: "FECHA Y HORA ISO 8601 CON ZONA HORARIA"

REASON:
TYPE: STRING
MAXLENGTH: 500

INSURANCEID:
TYPE: STRING

APPOINTMENTRESPONSE:

TYPE: OBJECT
PROPERTIES:
APPOINTMENTID:
TYPE: STRING
EXAMPLE: "APT-20250814"
STATUS:
TYPE: STRING

ENUM: [CONFIRMED, PENDING, CANCELLED, NO_SHOW]
CONFIRMEDSlot:
TYPE: STRING
FORMAT: DATE-TIME
DOCTOR:
\$REF: "#/COMPONENTS/SCHEMAS/DOCTORSUMMARY"
LOCATION:
TYPE: STRING
CREATEDAt:
TYPE: STRING
FORMAT: DATE-TIME

DOCTORSUMMARY:
TYPE: OBJECT
PROPERTIES:
DOCTORId:
TYPE: STRING
FULLNAME:
TYPE: STRING
SPECIALTY:
TYPE: STRING
CONSULTINGROOM:
TYPE: STRING

SLOTList:
TYPE: OBJECT
PROPERTIES:
DOCTORId:
TYPE: STRING
DATE:
TYPE: STRING
FORMAT: DATE
AVAILABLESLOTS:
TYPE: ARRAY
ITEMS:
TYPE: STRING
FORMAT: DATE-TIME

PROBLEMDETAIL:
TYPE: OBJECT
PROPERTIES:
TYPE:
TYPE: STRING
TITLE:
TYPE: STRING
STATUS:
TYPE: INTEGER
DETAIL:
TYPE: STRING
INSTANCE:
TYPE: STRING

RESPONSES:
BADREQUEST:

```
DESCRIPTION: "SOLICITUD MAL FORMADA"
CONTENT:
APPLICATION/PROBLEM+JSON:
SCHEMA:
  $REF: "#/COMPONENTS/SCHEMAS/PROBLEMDETAIL"
CONFLICT:
DESCRIPTION: "EL SLOT SOLICITADO YA NO ESTÁ DISPONIBLE"
CONTENT:
APPLICATION/PROBLEM+JSON:
SCHEMA:
  $REF: "#/COMPONENTS/SCHEMAS/PROBLEMDETAIL"
NOTFOUND:
DESCRIPTION: "RECURSO NO ENCONTRADO"
CONTENT:
APPLICATION/PROBLEM+JSON:
SCHEMA:
  $REF: "#/COMPONENTS/SCHEMAS/PROBLEMDETAIL"
SERVICEUNAVAILABLE:
DESCRIPTION: "MOTOR DE AGENDA TEMPORALMENTE NO DISPONIBLE"
CONTENT:
APPLICATION/PROBLEM+JSON:
SCHEMA:
  $REF: "#/COMPONENTS/SCHEMAS/PROBLEMDETAIL"

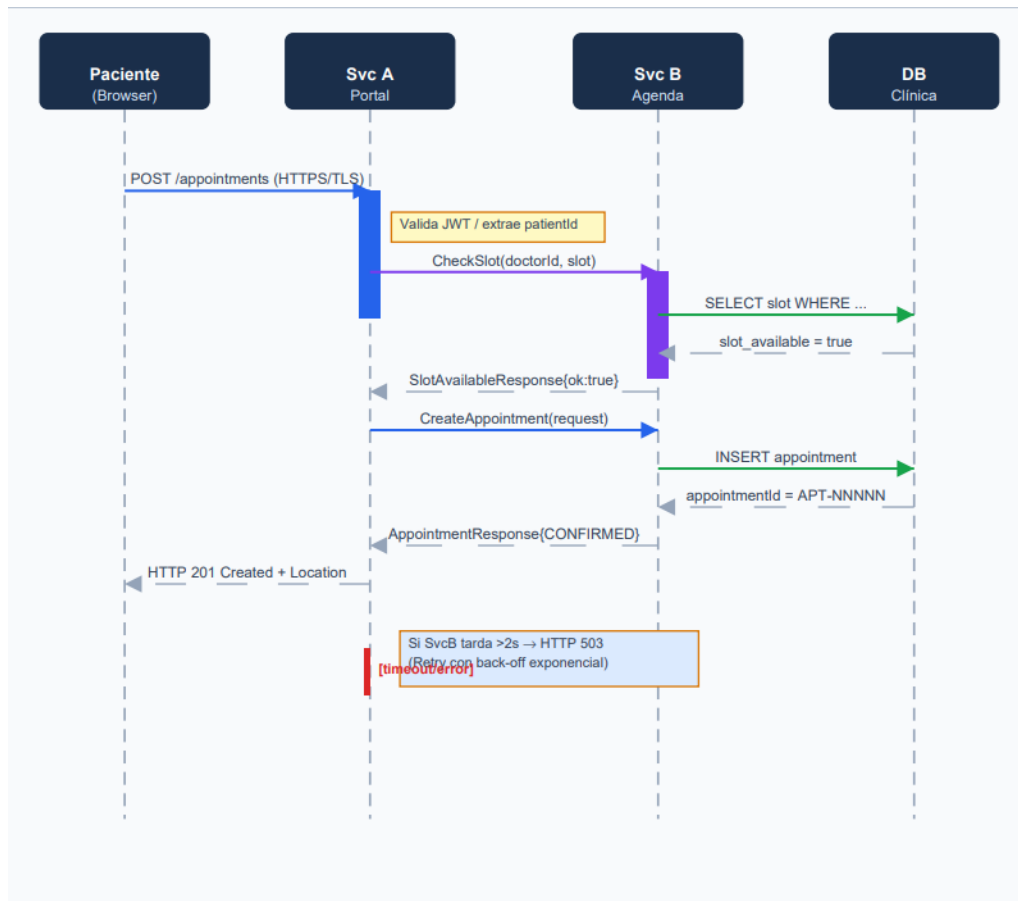
SECURITYSCHEMES:
BEARERAUTH:
  TYPE: HTTP
  SCHEME: BEARER
  BEARERFORMAT: JWT

SECURITY:
- BEARERAUTH: []
```

Diagrama de Secuencia

Flujo Normal

El siguiente diagrama muestra la interacción entre el Portal del Paciente (Svc A) y el Motor de Agenda (Svc B) durante una reserva exitosa. Se incluyen los timeouts y el manejo de errores de red según las falacias de la computación distribuida.



Cuadro Comparativo: REST vs. gRPC

La siguiente tabla sintetiza el análisis arquitectónico aplicado específicamente al sistema de reserva de citas médicas.

Dimensión	REST (JSON / HTTP 1.1)	gRPC (Protobuf / HTTP 2)
Protocolo de transporte	HTTP/1.1 — una solicitud por conexión TCP; keep-alive mejora pero no elimina la limitación	HTTP/2 — multiplexación de streams; múltiples RPCs concurrentes sobre una sola conexión TCP
Serialización & payload	JSON — texto plano y legible. Un AppointmentRequest típico ocupa ~420 bytes	Protocol Buffers — binario. El mismo mensaje ocupa ~110 bytes (74% menos)
Latencia	Mayor: headers HTTP repetidos en cada solicitud, sin compresión nativa de headers	Menor: headers HPACK comprimidos, sin overhead de parseo JSON, framing binario eficiente
Contrato / acoplamiento	OpenAPI es descriptivo; el cliente puede ignorar campos nuevos sin romperse (loose coupling)	.proto es prescriptivo; cliente y servidor deben compartir la misma versión del archivo (tight coupling)
Facilidad de depuración	Excelente: curl, Postman, DevTools del navegador. JSON es directamente legible por humanos	⚠️Difícil: binario no legible sin herramientas adicionales (grpcurl, Evans, plugin de Wireshark)
Streaming	No nativo; requiere WebSockets o SSE para notificaciones de slots en tiempo real	Nativo: server-streaming para WatchAvailableSlots(), sin infraestructura extra
Soporte de navegadores	Nativo en todos los navegadores; ideal para el	Requiere gRPC-Web + proxy (Envoy); agrega

	Portal del Paciente (React/Angular)	complejidad operativa al frontend
Manejo de errores	Códigos HTTP estándar (400, 409, 503) + RFC 7807 Problem Details; ampliamente conocidos	Status codes propios de gRPC (NOT_FOUND, UNAVAILABLE, ALREADY_EXISTS); semántica más granular pero menos universal
Generación de clientes	openapi-generator soporta 50+ lenguajes, aunque el código generado puede no ser idiomático	protoc genera stubs altamente eficientes y con tipado fuerte en Go, Java, Python, Node.js, etc.
Versionado	Versión en la URL (/v1/, /v2/); puede llevar a proliferación de endpoints con el tiempo	Backward-compatible por diseño: campos nuevos son opcionales; campos eliminados usan reserved
Seguridad	TLS + JWT en Authorization header; OAuth 2.0 / OIDC es el estándar ampliamente adoptado	mTLS nativo en el canal; interceptors para autenticación por RPC; mayor control pero más configuración

Discusión

Latencia y Carga Útil (Payload)

En el escenario de citas médicas, el tiempo de respuesta percibido por el paciente incluye:

latencia de red, serialización, procesamiento de la BD y la deserialización en el cliente. Protocol Buffers reduce la serialización de ~420 bytes (JSON) a ~110 bytes por mensaje, un ahorro del 74 %. En redes móviles con ancho de banda limitado (escenario frecuente en Ecuador: 4G con alta latencia), esta reducción de payload puede representar hasta 80 ms de diferencia por solicitud. Sin embargo, el overhead de parseo JSON en Node.js (V8) es suficientemente pequeño para mensajes pequeños (< 1 KB) que el beneficio de Protobuf sólo se vuelve significativo cuando la frecuencia de llamadas es alta (> 500 req/s) o cuando se activa el streaming de slots. Para la interfaz pública (Portal → API Gateway), REST con compresión gzip es perfectamente adecuado.

Facilidad de Depuración

REST gana de forma clara: cualquier desarrollador puede usar curl o Postman para reproducir una llamada fallida en producción. Los logs de un error 409 (slot ya ocupado) son inmediatamente comprensibles. Con gRPC, el equipo necesita instalar grpcurl o el plugin de Wireshark para decodificar el binario Protobuf. En un equipo clínico donde los desarrolladores pueden tener experiencia mixta, este costo de aprendizaje no es trivial.

La solución híbrida adoptada en esta arquitectura es usar gRPC-Gateway: un proxy que expone los métodos gRPC también como endpoints REST/JSON, manteniendo la eficiencia interna con Protobuf y ofreciendo depuración amigable para el equipo.

Acoplamiento entre Cliente y Servidor

Este es el trade-off más crítico del diseño. REST es loosely coupled: el cliente puede ignorar campos nuevos en el JSON de respuesta sin romperse. gRPC es fuertemente acoplado por diseño: si el equipo de SvcB modifica el archivo .proto sin seguir las reglas de evolución (no reutilizar números de campo, no cambiar tipos, usar reserved para campos eliminados), el stub generado en SvcA quedará incompatible. Para el sistema de citas médicas, donde los contratos de datos son regulados (RESO, HL7 FHIR) y los cambios son infrecuentes pero costosos, el acoplamiento fuerte de gRPC es en realidad una ventaja: cualquier cambio incompatible falla en tiempo de compilación, no en producción a las 2:00 AM.